



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 3
по курсу
«Теория игр в системах защиты информации»

Студент ИУ8-22М
 (Группа)

Е. В. Умеренко
(И. О. Фамилия)

Преподаватель:

П. В. Захаров
(И.О. Фамилия)

Цель работы

Найти оптимальные стратегии непрерывной выпукло-вогнутой антагонистической игры аналитическим и численным методами.

Постановка задачи

Для игры, заданной матрицей стратегий c_{ij} , требуется найти оптимальные смешанные стратегии обоих игроков, сведя матричную игру к задаче ЛП (прямой для одного игрока и двойственной для другого). Задачи ЛП следует решать симплекс-методом, приведя начальные, промежуточные и конечные симплекс-таблицы. Также по окончании алгоритма полученные решения необходимо проверить на допустимость.

Ход работы

Пусть задана игра:

a	b	c	d	e
-4	4	8	-12/5	-28/5

Аналитическое решение. Функция ядра имеет вид:

$$H(x, y) = -4x^2 + 4y^2 + 8xy - \frac{12}{5}x - \frac{28}{5}y$$

Проверим выполнимость условий для принадлежности игры к классу выпукло-вогнутых:

$$H_{xx} = 2a = -8 < 0$$

$$H_{yy} = 2b = 8 > 0$$

Функция вогнута по x (первый игрок максимизирует) и выпукла по y (второй минимизирует), то есть $H_{xx} < 0$ и $H_{yy} > 0$, значит представленная игра выпукло-вогнутая и разрешима в чистых стратегиях. Для нахождения оптимальных стратегий найдем производные функции ядра по каждой переменной:

$$\frac{\partial H}{\partial x} = 2ax + cy + d = 0, \quad \frac{\partial H}{\partial y} = 2by + cx + e = 0.$$

Подставим коэффициенты:

$$-8x + 8y - \frac{12}{5} = 0, \quad 8y + 8x - \frac{28}{5} = 0.$$

Решим систему:

$$\begin{cases} y - x = 0.3, \\ y + x = 0.7. \end{cases}$$

Получаем:

$(x^*, y^*) = (0.2, 0.5)$, что принадлежит интервалу от 0 до 1.

$(x^*, y^*) = (0.2, 0.5)$ - седловая точка в чистых стратегиях.

Цена игры:

$$h = H(0.2, 0.5) = -4 \cdot 0.04 + 4 \cdot 0.25 + 8 \cdot 0.2 \cdot 0.5 - \frac{12}{5} \cdot 0.2 - \frac{28}{5} \cdot 0.5.$$

$$H(x, y) = -1.64$$

Численное решение. Для решения игры с непрерывным ядром используем метод аппроксимации функции выигрышей на сетке. С помощью программы найдем решения при различном шаге сетки для исходной задачи. Результат численного представлен ниже.

Ядро игры: $H(x, y) = -4 \cdot x^2 + 4 \cdot y^2 + 8 \cdot x \cdot y + -2.4 \cdot x + -5.6 \cdot y$

Оптимальная стратегия игрока А: $x^* = 0.200000$

Оптимальная стратегия игрока В: $y^* = 0.500000$

Цена игры: $v = -1.640000$

N=2

```
[
[ 0.000 -1.800 -1.600]
[ -2.200 -2.000 0.200]
[ -6.400 -4.200 0.000]
```

]

Есть седловая точка:

x=0.000 y=0.500 H=-1.800

N=3

```
[
[ 0.000 -1.422 -1.956 -1.600]
[ -1.244 -1.778 -1.422 -0.178]
[ -3.378 -3.022 -1.778 0.356]
[ -6.400 -5.156 -3.022 0.000]
]
```

Достигнут максимум итераций (200)

Седловой точки нет, решение методом Брауна-Робинсона:

x=0.333 y=0.333 H=-1.778

Потребовалось итераций: 200

N=4

```
[
[ 0.000 -1.150 -1.800 -1.950 -1.600]
[ -0.850 -1.500 -1.650 -1.300 -0.450]
[ -2.200 -2.350 -2.000 -1.150 0.200]
[ -4.050 -3.700 -2.850 -1.500 0.350]
[ -6.400 -5.550 -4.200 -2.350 0.000]
]
```

Есть седловая точка:

x=0.250 y=0.500 H=-1.650

N=5

```
[
[ 0.000 -0.960 -1.600 -1.920 -1.920 -1.600]
[ -0.640 -1.280 -1.600 -1.600 -1.280 -0.640]
[ -1.600 -1.920 -1.920 -1.600 -0.960 0.000]
[ -2.880 -2.880 -2.560 -1.920 -0.960 0.320]
[ -4.480 -4.160 -3.520 -2.560 -1.280 0.320]
[ -6.400 -5.760 -4.800 -3.520 -1.920 0.000]
]
```

Есть седловая точка:

x=0.200 y=0.400 H=-1.600

```
=====
Аналитическое решение:      x* = 0.2000, y* = 0.5000, v = -1.6400
Численное решение (N=5): x* = 0.2000, y* = 0.4000, v = -1.6000
=====
```

Таким образом, численно найдено решение задачи: $x = 0.2$ $y = 0.4$
 $H = -1.600$

Приложение А.

```
import numpy as np

class ContinuousGameSolver:
    def __init__(self, a, b, c, d, e):
        self.a = a
        self.b = b
        self.c = c
        self.d = d
        self.e = e

    def H(self, x, y):
        return self.a*x**2 + self.b*y**2 + self.c*x*y + self.d*x + self.e*y

    def analytical_solution(self):
        A1, B1, C1 = 2*self.a, self.c, -self.d
        A2, B2, C2 = self.c, 2*self.b, -self.e

        det = A1*B2 - A2*B1

        x_star = (C1*B2 - C2*B1) / det
        y_star = (A1*C2 - A2*C1) / det

        x_star = max(0, min(1, x_star))
        y_star = max(0, min(1, y_star))

        v_star = self.H(x_star, y_star)

        print(f"\nЯдро игры:  $H(x,y) = \{self.a\} \cdot x^2 + \{self.b\} \cdot y^2 +$   

 $\{self.c\} \cdot x \cdot y + \{self.d\} \cdot x + \{self.e\} \cdot y$ ")
        print(f"\nОптимальная стратегия игрока А:  $x^* = \{x\_star:.6f\}$ ")
        print(f"Оптимальная стратегия игрока В:  $y^* = \{y\_star:.6f\}$ ")
        print(f"Цена игры:  $v = \{v\_star:.6f\}$ ")

        return x_star, y_star, v_star

    def create_payoff_matrix(self, N):
        x_points = np.linspace(0, 1, N+1)
        y_points = np.linspace(0, 1, N+1)

        matrix = np.zeros((N+1, N+1))
        # Заполнение матрицы значениями ядра
        for i, x in enumerate(x_points):
            for j, y in enumerate(y_points):
                matrix[i, j] = self.H(x, y)

        return matrix, x_points, y_points
```

```

def solve_by_brown_robinson(self, matrix, max_iter=200, eps=1e-6):

    m, n = matrix.shape

    x_counts = np.zeros(m)
    y_counts = np.zeros(n)
    x_counts[0] = 1
    y_counts[0] = 1

    prev_x_star = None
    prev_y_star = None
    prev_x_idx = None
    prev_y_idx = None
    consecutive_small_errors = 0

    for k in range(1, max_iter + 1):
        x_emp = x_counts / k
        y_emp = y_counts / k

        expected_gains = np.dot(matrix, y_emp)
        best_i = np.argmax(expected_gains)

        expected_losses = np.dot(x_emp, matrix)
        best_j = np.argmin(expected_losses)

        x_counts[best_i] += 1
        y_counts[best_j] += 1

        x_star = x_counts / (k + 1)
        y_star = y_counts / (k + 1)
        x_idx = np.argmax(x_star)
        y_idx = np.argmax(y_star)

        if prev_x_star is not None and prev_y_star is not None:
            error = (abs(x_star[x_idx] - prev_x_star[prev_x_idx]) +
                    abs(y_star[y_idx] - prev_y_star[prev_y_idx]))

            if error < eps:
                consecutive_small_errors += 1
                if consecutive_small_errors >= 5:
                    print(f" Остановка на итерации {k}: ошибка < {eps}
в течение 5 итераций подряд")
                    return x_idx, y_idx, k
            else:
                consecutive_small_errors = 0

        prev_x_star = x_star.copy()
        prev_y_star = y_star.copy()
        prev_x_idx = x_idx
        prev_y_idx = y_idx

```

```

print(f" Достигнут максимум итераций ({max_iter})")
return x_idx, y_idx, max_iter

def numerical_solution(self, max_N=12):
    results = []

    for N in range(2, max_N + 1):
        matrix, x_points, y_points = self.create_payoff_matrix(N)

        print(f"\nN={N}")
        print("[")
        for i in range(N+1):
            row_str = "["
            for j in range(N+1):
                row_str += f"{matrix[i,j]:7.3f}"
                if j < N:
                    row_str += " "
            row_str += "]"
            print(row_str)
        print("]")

        row_mins = np.min(matrix, axis=1)
        lower_price = np.max(row_mins)
        col_maxs = np.max(matrix, axis=0)
        upper_price = np.min(col_maxs)

        saddle_found = False
        saddle_i, saddle_j = -1, -1

        for i in range(N+1):
            for j in range(N+1):
                if (abs(matrix[i,j] - row_mins[i]) < 1e-6 and
                    abs(matrix[i,j] - col_maxs[j]) < 1e-6):
                    if abs(lower_price - upper_price) < 1e-6:
                        saddle_found = True
                        saddle_i, saddle_j = i, j
                        break
            if saddle_found:
                break

        if saddle_found:
            x_num = x_points[saddle_i]
            y_num = y_points[saddle_j]
            v_num = matrix[saddle_i, saddle_j]
            print(f"\nЕсть седловая точка:")
            print(f"x={x_num:.3f} y={y_num:.3f} H={v_num:.3f}")
            results.append({
                'N': N, 'method': 'седловая точка',
                'x': x_num, 'y': y_num, 'v': v_num, 'iterations': None
            })

```

```

        })
    else:
        x_idx, y_idx, iterations =
self.solve_by_brown_robinson(matrix)
        x_num = x_points[x_idx]
        y_num = y_points[y_idx]
        v_num = matrix[x_idx, y_idx]
        print(f"\nСедловой точки нет, решение методом Брауна-
Робинсона:")
        print(f"x={x_num:.3f} y={y_num:.3f} H={v_num:.3f}")
        print(f" Потребовалось итераций: {iterations}")
        results.append({
            'N': N, 'method': 'Брауна-Робинсона',
            'x': x_num, 'y': y_num, 'v': v_num, 'iterations':
iterations
        })

    return results

def main():
    a, b, c, d, e = -4, 4, 8, -12/5, -28/5

    solver = ContinuousGameSolver(a, b, c, d, e)

    x_ana, y_ana, v_ana = solver.analytical_solution()

    numerical_results = solver.numerical_solution(max_N=5)
    print(f"\n{'='*60}")
    print(f"Аналитическое решение:      x* = {x_ana:.4f}, y* = {y_ana:.4f}, v
= {v_ana:.4f}")

    best = numerical_results[-1]
    print(f"Численное решение (N={best['N']}): x* = {best['x']:.4f}, y* =
{best['y']:.4f}, v = {best['v']:.4f}")
    if best['iterations']:
        print(f" Итераций потребовалось: {best['iterations']}")
    print(f"\n{'='*60}")

```